TOTEM INTERACTIVE

INNOVATE. DESIGN. EXPLORE

Full Stack Developer Assignment

Voice-Driven Deterministic Prompt Optimization Engine

01 — Purpose

The main challenge is to build a system that takes **raw voice or unstructured input (including rants, mixed language, unclear thoughts)** and converts it into a **minimum viable prompt (Minimum VIABLE Prompt)**.

The system must:

- Automatically understand and extract the user's intent
- Confirm the intent According to voice input with the user before execution
- Prompt engineer the input into a **clean, minimal, and usable prompt**
- Ensure the output is **token-efficient and not verbose (CAVEMAN MODE)**

The system should behave consistently for similar inputs and should not rely on manual intervention unless necessary.

Bonus (Optional):
 If you can show how the system improves prompts using prior context or memory during chat.

02 — Problem Statement

Key Challenges

1. Voice to Intent

- Speech-to-text inaccuracies
- Multilingual inconsistencies
- Lack of validation

2. Intent Interpretation

- Ambiguous inputs
- No confirmation before execution

3. Token Compression / Optimisation

- Verbose prompts
- Context window wastage

03 — Key Objectives

- Build a multilingual voice-to-prompt pipeline
- Implement intent confirmation before execution
- Develop a structured prompt transformation
- Generate minimal, high-signal outputs
- Reduce token usage without compromising accuracy
- Ensure deterministic system behavior **(need a frontend display for this)**

04 — System Architecture

Design your system architecture.

Requirement:

- Must be submitted as a Mermaid Diagram
- Should clearly show:
    - Input flow
    - Processing layers
    - Decision points
    - Output generation

05 — System Modules

5.1 Voice Input Module

- Accept English, Hindi, Hinglish
- Handle noisy input

5.2 Speech-to-Text Module

- Convert voice → text
- Detect language + confidence score

5.3 Confidence Handling

- Handle unclear or low-confidence input
- The approach is up to you

5.4 Intent Detection & Confirmation (Critical Layer)

System must:

- Extract intent
- Present interpreted intent to the user
- Require confirmation before proceeding
- No confirmation → No execution

Confirmation Step:

"You want [task] with [format]. Confirm?"

5.5 Prompt ENHACEMENT and Decomposition

```
{
 "intent": "",
 "task": "",
```

```
 "domain": "",
 "constraints": [],
 "output_format": "",
 "audience": ""
}
```

5.6 Prompt Transformation Engine

- Role definition
- Constraint injection
- Context enrichment
- Output formatting

Rule: Generate only one optimized prompt

5.7 Token Optimization / Minimum Viable Prompt

System must:

- Convert raw or voice input into a minimum viable prompt
- Remove all unnecessary tokens
- Ensure output is directly usable by an LLM

5.8 Validation Layer

- Intent alignment
- Format correctness
- Token efficiency

06 — Failure Handling & Recovery

System should handle:

- Unclear input
- Incorrect intent
- Processing failures

The approach is up to you.
Focus on system reliability.

07 — Noise & Abuse Filtering

Noise Handling

- Remove filler words
- Clean repetition

Guardrails

- Reject non-task inputs
- Enforce actionable prompts

Language Normalization

- Convert all input → English

08 — Functional Requirements

- Multilingual voice input
- Intent validation
- Structured prompt generation
- Deterministic enhancement
- Minimal output generation
- 30–50% token reduction

09 — Evaluation Criteria

Criteria	Priority
Intent Accuracy	Critical
Voice Handling	High
Output Minimalism	Critical
Token Efficiency	High
System Design	High
Front End Design	Medium

---

11 — Example Flow

Input:
"Ek marketing plan bana do for gym app."

Output:

Create a 3-step marketing plan for a gym app.
Output: bullet points
Constraint: under 100 words

12 — Deliverables

Code

- GitHub repository
- Clean modular structure

Application

- Working on a frontend chat application

System Components

- Smart memory allocator logic
- Graph visualization page
- Database schema + initialization script

Demo Video (2–3 min)

Must include:

- Code in a GitHub repo.
- Database schema + init script.
- README with setup instructions and example API calls.
- Chat interaction
- Saving vs skipping
- Memory creation
- Memory update/merge
- Graph visualization
- Decision logs
- Final memory output (cards/logs)

13 — Submission Guidelines

Submit to: career@toteminteractive.in / Totemistaken@gmail.com / Shoeb@toteminteractive.in / Arjungujar@toteminteractive.in